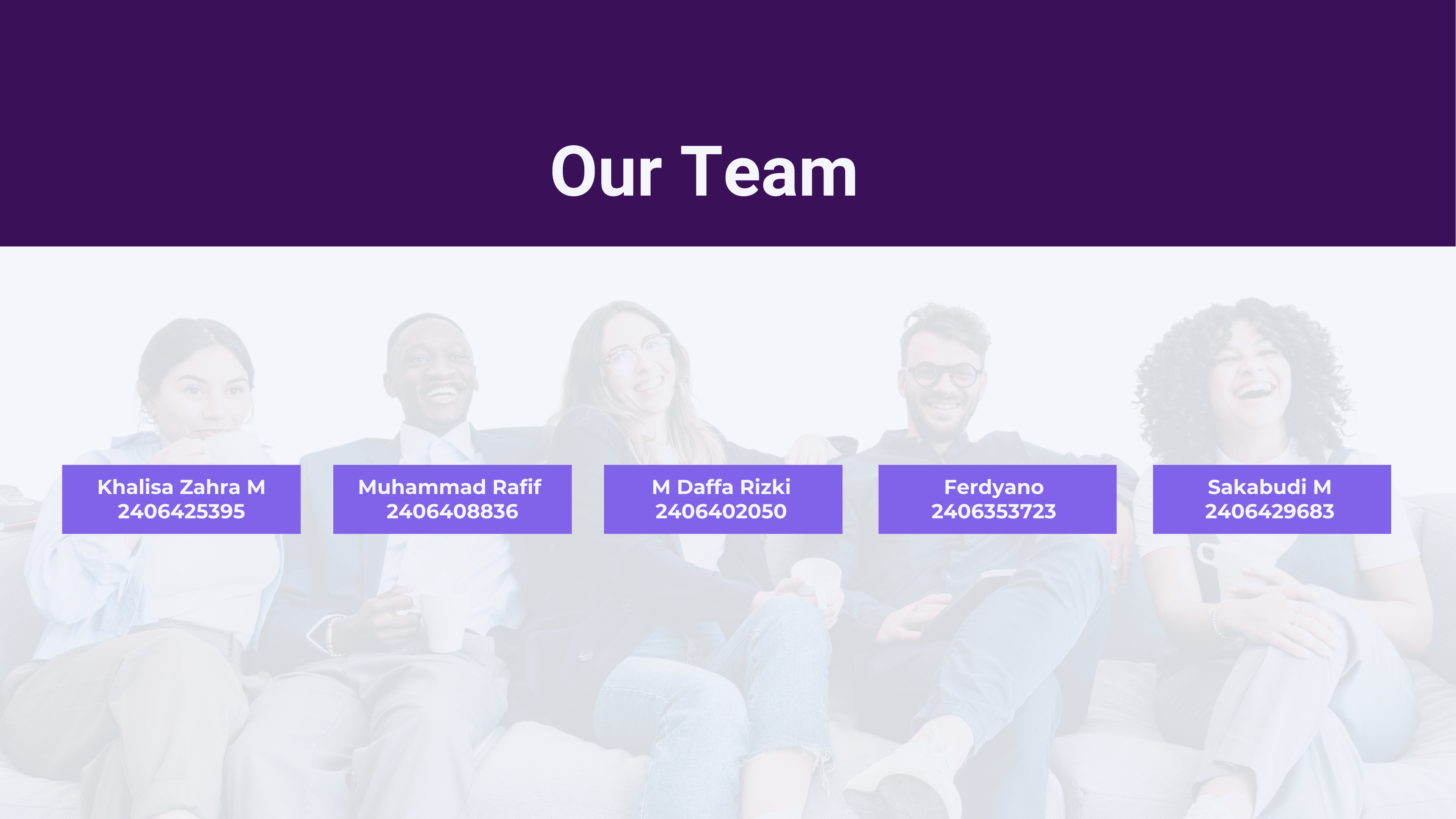


Kelompok 5
Siang

MINI PROJECT

MONGODB

Our Team



Khalisa Zahra M
2406425395

Muhammad Rafif
2406408836

M Daffa Rizki
2406402050

Ferdyano
2406353723

Sakabudi M
2406429683

COMPANY PROFILE



Bosch Group didirikan 1889 merupakan supplier teknologi global, memiliki 14B+ connected devices (proyeksi global 2022). Divisi Bosch digital (Bosch Software Innovatopns) mengoprasikan platform Bosch IoT insights yaitu layanan SaaS yang mengumpulkan dan menganalisis data dari jutaan sensor industri lintas sektor automative, energy, agriculture, dan retail

PERMASALAHAN DATA IOT :

- **Volume & kecepatan data**, sensor (milidetik dari ribuan mesin, skala melampaui kapasitas polling)
- **Heterogenitas format data**, setiap mesin menghasilkan skema data yang berbeda
- **Realtime analysis**, latensi menit/jam tidak diterima safety critical applications
- **Multi-cloud portability**, pelanggan bosch perlu fleksibilitas hosting, berjalan di multiple cloud providers

CREATIVE APPLICATION

Problem:

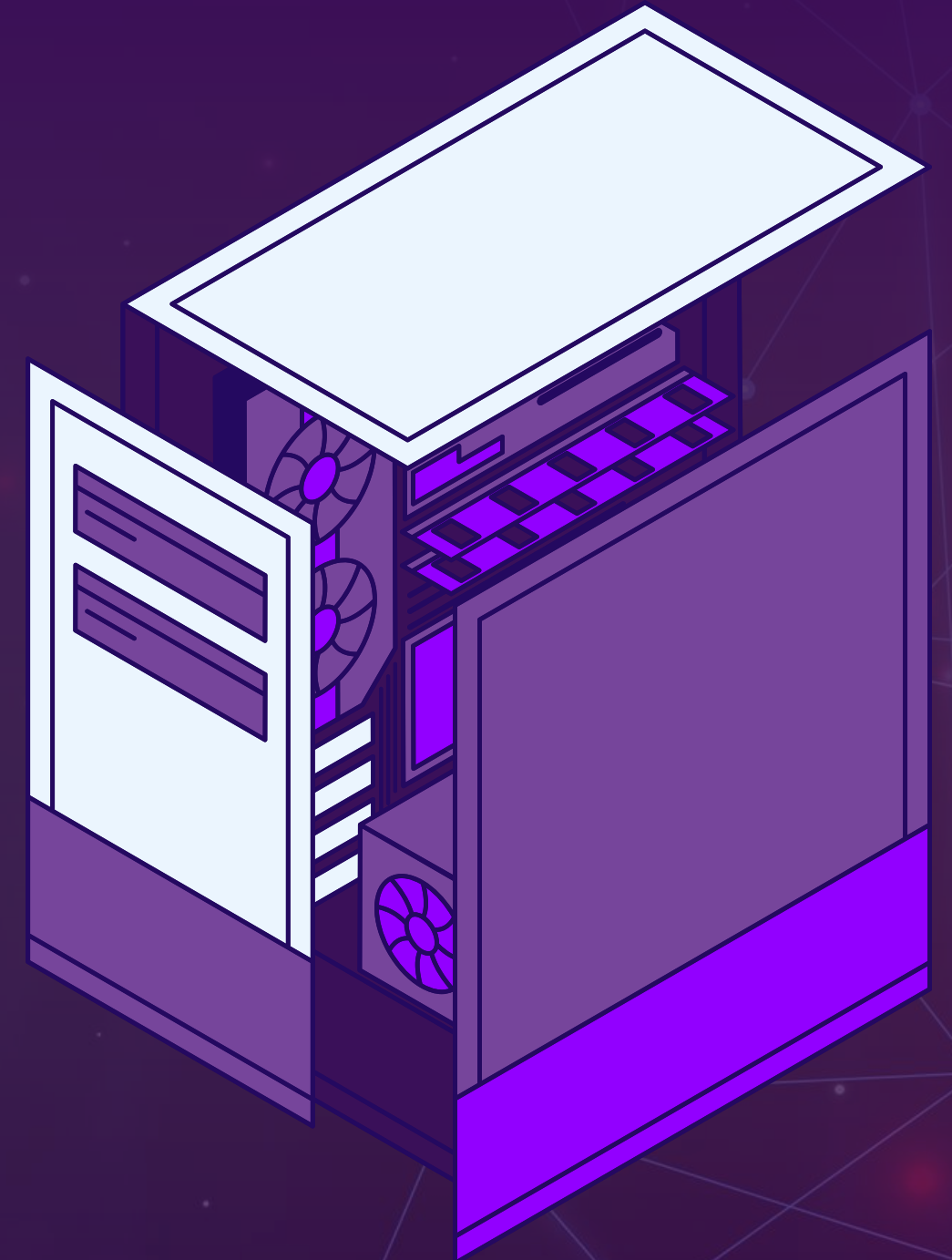
Data sensor IoT memiliki frekuensi yang tinggi (1 read/s)

Impact:

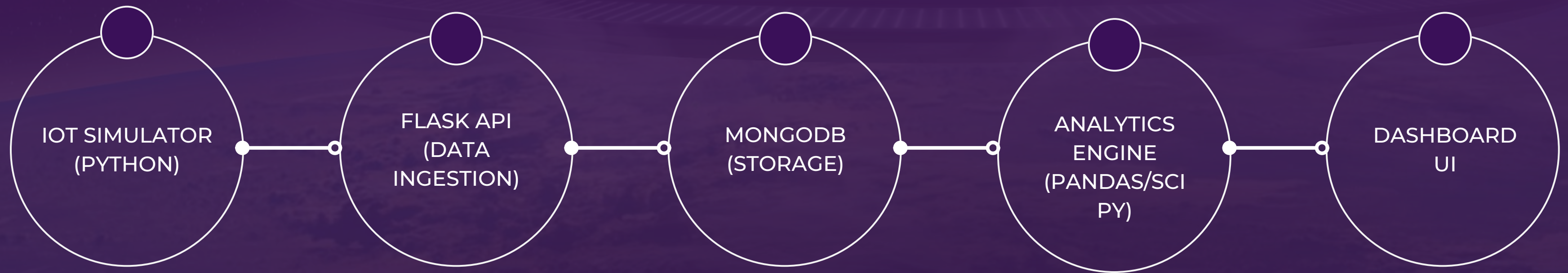
1 sensor = 86.400 data/hari. 100 sensor = 8,6 Juta data/hari. Database biasa akan mengalami *I/O bottleneck*.

Motivation:

Membangun Telemetry Engine yang bisa menerima data besar dan membuat data tersebut bisa dibaca secara mudah dan real-time.



SYSTEM ARCHITECTURE



APP INTERFACE



DATA MODELING

Kenapa MongoDB?

Jenis sensor industri

Pabrik modern menggunakan berbagai jenis sensor (temperature, vibration, pressure). Setiap sensor menghasilkan payload JSON dengan struktur (key-value) yang sama sekali berbeda.

Kelemahan database relasional

Saat menggunakan SQL setiap penambahan sensor baru akan memaksa administrator melakukan proses ALTER TABLE, pada pabrik yang berjalan 24/7 hal ini bisa menyebabkan system downtime.

Keunggulan Schema-less

MongoDB memungkinkan penyimpanan data vibration dan data temperature di dalam satu collection yang sama secara langsung (Native JSON format) tanpa memerlukan definisi kolom statis di awal.

BUCKET PATTERN

Problem

Menyimpan 1 data sensor sebagai 1 dokumen secara individual dengan frekuensi 1 Hz (1 data per detik), 1 sensor menghasilkan 86.400 dokumen/hari. Jika ada banyak sensor ini akan bermasalah pada kapasitas memory yang ada.

Solution

Mengelompokkan data ke dalam "Bucket" berbasis waktu dimana 1 bucket menyimpan data selama 5 menit dengan kapasitas maksimal: 300 data readings.

BUCKET PATTERN

Bucket dibuat di memori dengan Compound ID untuk mencegah adanya duplikasi dan array kosong measurements akan dibuat untuk menampung data time-series dari sensor.

```
new_bucket = {  
    "_id": bucket_id,  
    "motor_id": motor_id,  
    "bucket_start": now,  
    "count": 0,  
    "measurements": [],  
    "stats": {  
        "vib_rms": None,  
        "temp_avg": None,  
        "temp_max": None  
    },  
    "is_closed": False  
}
```

HIGH-THROUGHPUT INGESTION

Problem

Melakukan proses update dokumen sebanyak 300 kali per 5 menit berisiko memberatkan server jika harus mengganti seluruh dokumen setiap detiknya.

Solution

Memanfaatkan operasi Atomic Modifier \$push dan \$inc bawaan MongoDB maka backend flask tidak perlu load array berukuran besar ke memori aplikasi dan operasi push data (\$push) terjadi didalam engine database ingestion rate tetap tinggi.

```
buckets_col.update_one(  
    {"_id": bucket_id},  
    {  
        "$push": {"measurements": measurement},  
        "$inc": {"count": 1}  
    }  
)
```

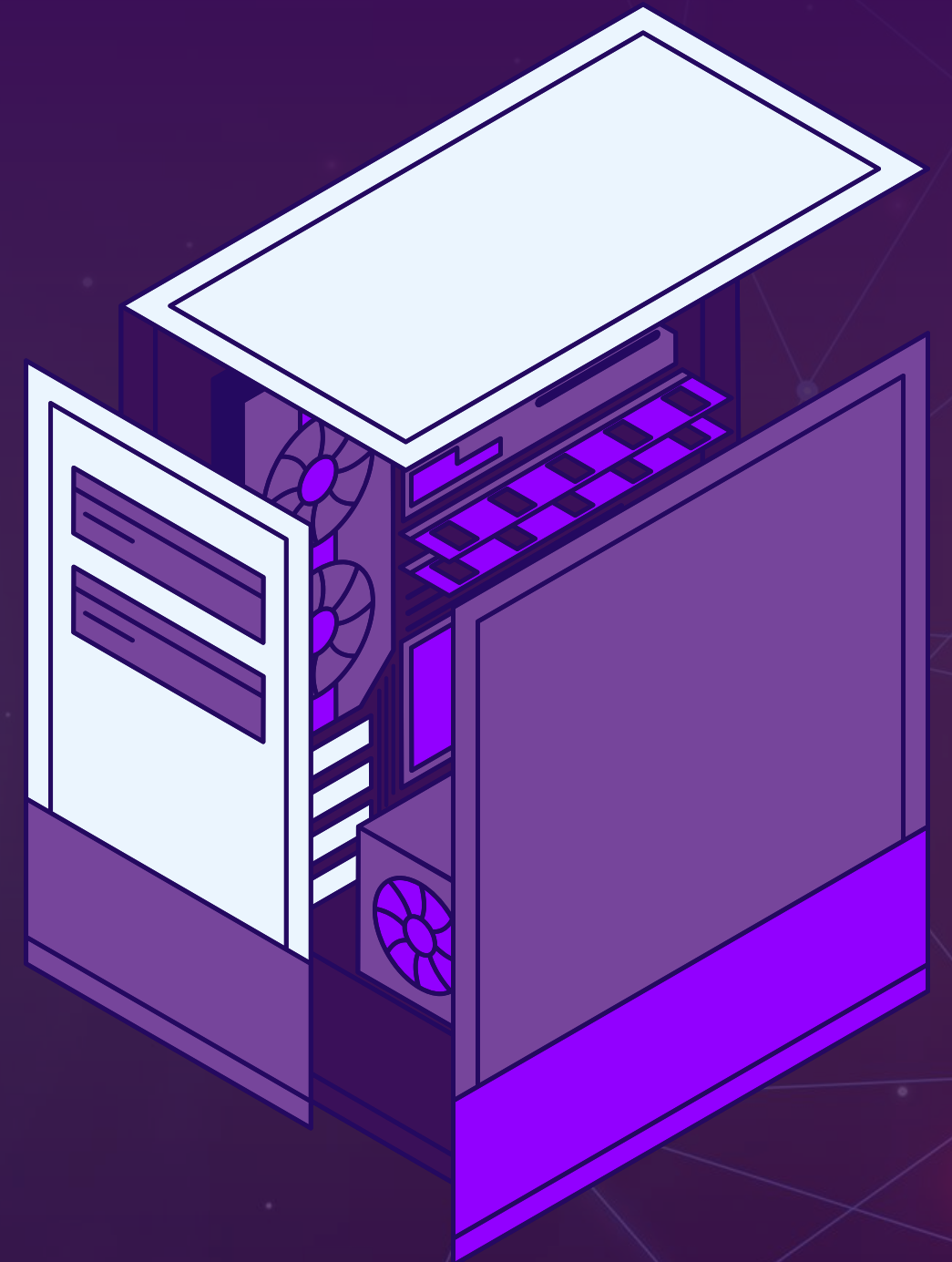

PRE-AGGREGATION

Pre-Aggregation: Menggeser beban komputasi dari proses read ke proses write dengan menghitung data di background agar data bisa ditampilkan saat dibutuhkan oleh UI dashboard. Tujuannya adalah agar dashboard Streamlit tidak perlu menghitung ulang raw array data untuk menampilkan grafik average dan cukup membaca nilai `stats.temp_avg`.

```
stats = {  
    "vib_rms": round(rms, 4),  
    "temp_avg": round(sum(temps) / len(temps), 2),  
    "temp_max": round(max(temps), 2)  
}  
  
buckets_col.update_one(  
    {"_id": bucket_id},  
    {"$set": {"is_closed": True, "stats": stats}}  
)
```

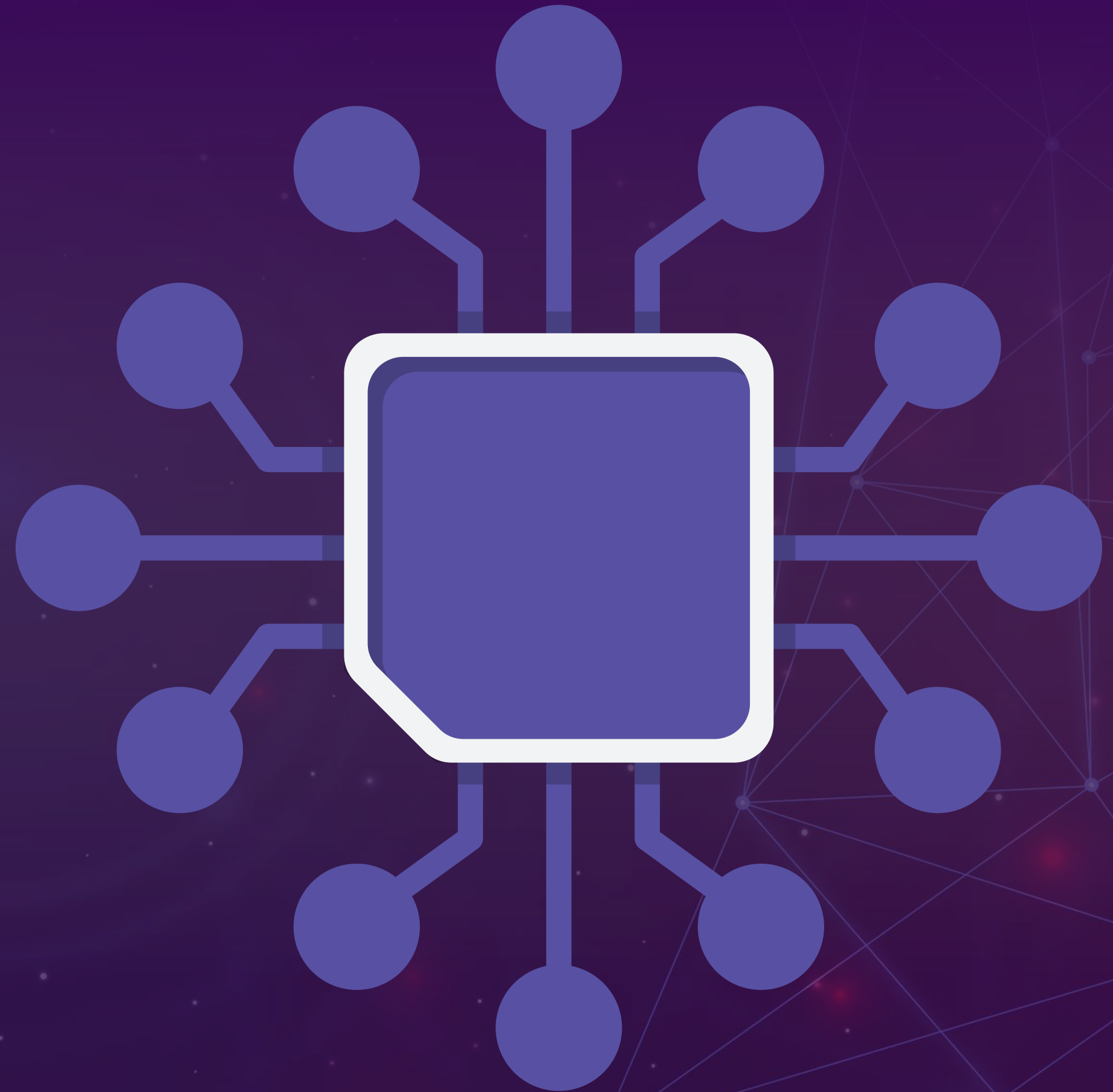
BENCHMARK_NEONDB.PY

File `benchmark_neondb.py` merupakan script pengujian otomatis untuk mengevaluasi dan membandingkan latency antara Local Flask API (NoSQL) dengan database Cloud PostgreSQL (NeonDB). Script ini bekerja dengan menghasilkan 300 data sensor dummy yang mensimulasikan metric mesin seperti acceleration, gyroscope, temprature, dan humidity, lalu mengukur waktu execution secara. Script akan menguji dua operasi inti IoT: waktu yang dibutuhkan untuk menulis data satu per satu dan waktu yang dibutuhkan untuk menarik kembali seluruh 300 rekaman data tersebut dari kedua database.



LIST ENDPOINT

- **POST /telemetry**
- **GET /buckets/<motor_id>**
- **GET /health/<motor_id>**
- **GET /analytics/<motor_id>**



ANALYTICS ENGINE

Array measurements yang ditarik dari MongoDB dapat diinjeksi langsung tanpa diubah menjadi pandas dataframe (pd.DataFrame(measurements_list)) untuk memudahkan deteksi Z-Score anomaly dan analisis spektrum frekuensi getaran.

```
#konversi raw signal to list of freq num & mag
def fft_spectrum(df, axis_column='az', sample_rate=50):
    if df.empty:
        return {
            "frequencies": [],
            "magnitudes": []
        }

    N = len(df)

    if N<2:
        return {
            "frequencies": [],
            "magnitudes": []
        }

    y_freq = fft(df[axis_column].values)
    x_freq = fftfreq(N, 1/sample_rate)

    #konversi bil. i ke real
    positive_frequencies = x_freq[:N//2]
    magnitudes = np.abs(y_freq[0:N//2])

    broken_component = "Normal"
    max_idx = np.argmax(magnitudes)
    dominant_freq = positive_frequencies[max_idx]
    max_mag =magnitudes[max_idx]
```

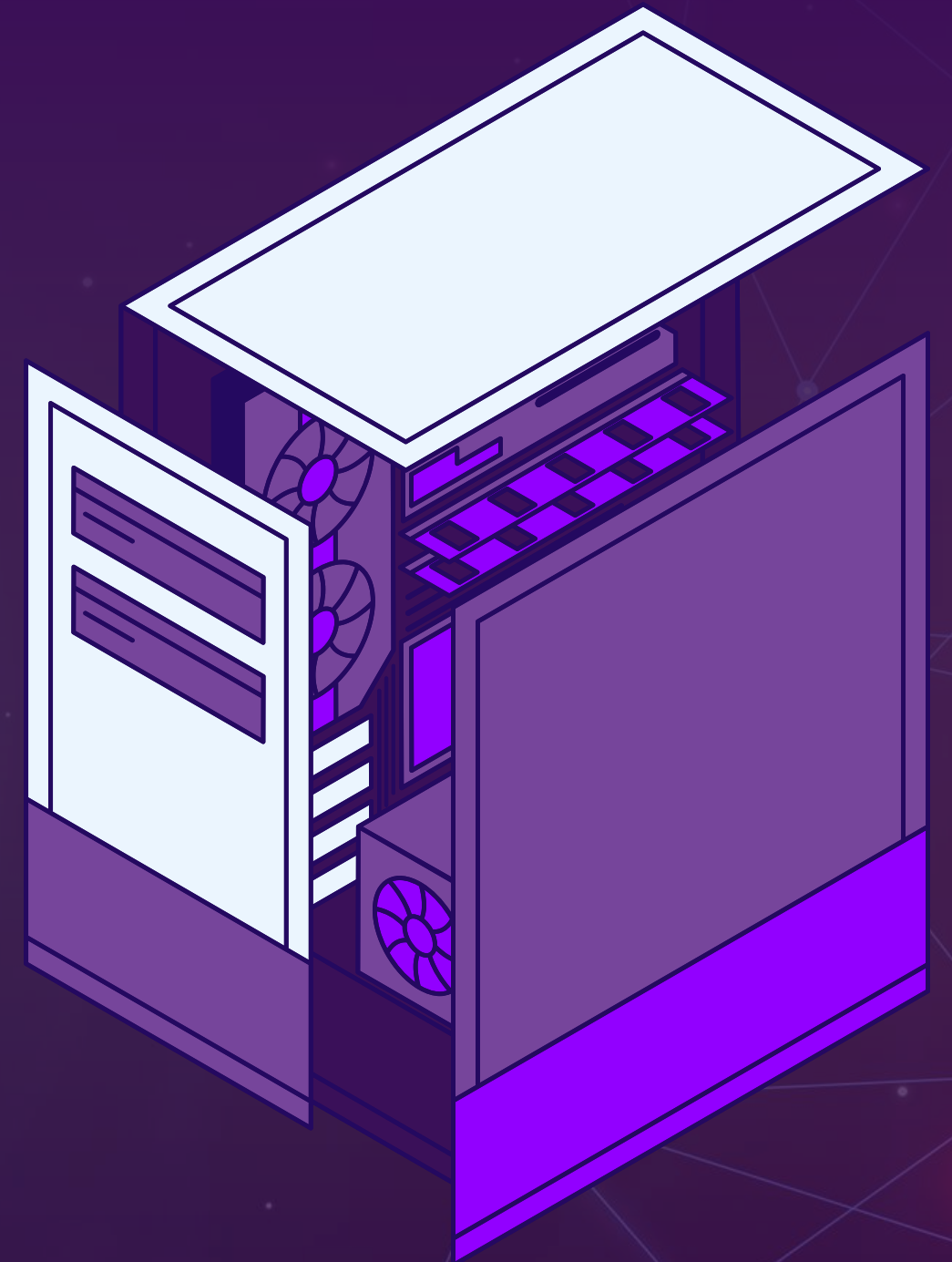
```
if max_mag>5.0:
    if 20<= dominant_freq <= 30:
        broken_component="Rotor"
    elif 45 <= dominant_freq <= 55:
        broken_component="Mechanical clearance/Stator electricity"
    elif dominant_freq > 100:
        broken_component="Bearing"

return {"frequencies": positive_frequencies.tolist(),
        "magnitudes": magnitudes.tolist(),
        "detected_fault": broken_component,
        "dominant_freq": float(dominant_freq)
}
```


TRADE-OFF: READ SPEED VS WRITE COMPLEXITY

Pro: Kecepatan agregasi data untuk dashboard (Read) meningkat dan proses load grafik tren suhu dan analitik spektrum pada dashboard IndustryOS dapat terjadi tanpa delay dan loading lama.

Con: Logika di sisi aplikasi (Write) menjadi jauh lebih complex dibandingkan sistem CRUD biasa dan juga kompleksitas dari backend.



SQL



NoSQL

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
127.0.0.1 - - [22/May/2026 20:09:33] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:33] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:33] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:34] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:35] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:09:36] "POST /telemetry HTTP/1.1" 201 -
127.0.0.1 - - [22/May/2026 20:10:01] "GET /buckets/MOTOR_BENCHMARK HTTP
/1.1" 200 -

Read Winner: NoSQL (Faster)

C:\Users\mdaff\Documents\SBD-KELAS\ProyekSBDKelas-BoschMonitor\backend>py bench
mark_neondb.py
Connecting to NeonDB (Cloud PostgreSQL)...
v NeonDB ready. The measurements table has been reset.

=== STARTING BENCHMARK: 300 SENSOR DATA ===
Comparing Local API Flask (NoSQL) vs Cloud NeonDB (SQL)

--- TEST 1: WRITE LATENCY ---
NoSQL (API Bucket Pattern) : 32051.80 ms
SQL (NeonDB Row-Based) : 25885.43 ms
Write Winner: SQL (Faster)

--- TEST 2: READ LATENCY ---
NoSQL (API Bucket Pattern) : 30.10 ms
SQL (NeonDB Row-Based) : 135.66 ms
Read Winner: NoSQL (Faster)
```

Berdasarkan hasil benchmark yang menguji 300 data sensor, terdapat perbandingan performa (trade-off) yang sangat jelas antara pendekatan NoSQL (Local API Bucket Pattern) dan SQL (Cloud NeonDB). Pada operasi penulisan data (Write Latency), database SQL mengungguli NoSQL. Hal ini menunjukkan bahwa metode insert database secara langsung berbasis baris (row-based) sangat efisien untuk pencatatan data yang berurutan. Namun sebaliknya, pada operasi pengambilan data (Read Latency), pendekatan NoSQL menggunakan Bucket Pattern terbukti jauh lebih cepat. Ini membuktikan bahwa pengelompokan data ke dalam bucket sangat optimal untuk menarik data telemetri dalam jumlah besar sekaligus, menjadikannya arsitektur yang ideal untuk memuat data historis secara cepat pada dashboard pemantauan kami.

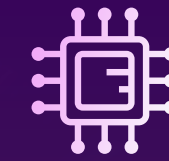


CONCLUSION



Penggunaan document store dengan bucket pattern adalah salah satu arsitektur best practice untuk menangani banyaknya Time-Series data pada industri IoT (IIoT). Sistem berhasil membuktikan efisiensi penyimpanan dan percepatan komputasi analitik untuk memvisualisasikan status mesin.

INDIVIDUAL CONTRIBUTIONS



Kelompok 5
Siang

Khalisa: Database Architect & Backend API Developer

- Merancang dan mengimplementasikan koneksi serta logika bucket MongoDB di file db.py.
- Mengatur struktur dokumen bucket untuk menampung ± 300 data pembacaan per 5 menit guna efisiensi query.
- Membangun backend menggunakan Flask API di file `app.py`.
- Membuat endpoint POST /api/v1/telemetry untuk menerima payload JSON dari skrip simulator, melakukan validasi skema, dan memanggil fungsi insert ke MongoDB.
- Menyiapkan endpoint GET /api/v1/health/{motor_id} untuk menyajikan skor kesehatan dan data terbaru.

Rafif: Data Analyst & Pipeline Engineer

- Fokus pada pemrosesan dan analisis data tingkat lanjut di file `analytics.py`.
- Mengimplementasikan pipeline analitik menggunakan Pandas, NumPy, dan SciPy.
- Membangun logika algoritma seperti RMS Vibration, rolling temperature trend, deteksi anomali Z-score, dan perhitungan FFT Spectrum untuk deteksi frekuensi kerusakan.
- Membuat endpoint GET /api/v1/analytics/{motor_id} untuk menampilkan skor predictive_health dari hasil analitik sebelumnya dan menampilkan hasil analitik tersebut.
- Membuat benchmark_neondb.py untuk perbandingan penggunaan SQL dan NoSQL

Daffa: IoT Simulation Engineer

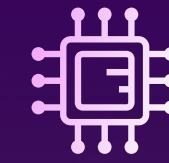
- Menulis skrip simulator menggunakan Python (simulator.py) untuk mengirim data telemetry JSON secara terus-menerus ke endpoint Flask.
- Mengatur logika dan interval pengiriman data dalam interval waktu tertentu.
- Membuat skenario simulasi dalam skrip untuk 3 kondisi operasi motor (normal, warning, fault) agar menghasilkan data anomali yang realistis secara otomatis.

Ferdyano: Frontend Dashboard Developer

- Membangun user interface yang interaktif menggunakan Streamlit di file dashboard.py.
- Mendesain visualisasi untuk halaman utama: Live Monitor (grafik real-time), Frequency Analysis, Anomaly Log, dan Maintenance Predictor.
- Menghubungkan visualisasi dasbor dengan endpoint API dari backend dan output data dari pipeline analitik.

Saka: Project Manager & Tech Writer

- Menyusun dokumentasi teknis sistem dan mengkoordinasikan laporan proyek.
- Membuat 15–20 slide presentasi (PPT) yang mencakup studi kasus Bosch, arsitektur sistem, pemodelan data, dan lessons learned.
- Mengelola repositori GitHub, menyusun README.md, serta memverifikasi histori commit Git.



GITHUB LINK

<https://github.com/lisamzhr/ProyekSBDKelas-BoschMonitor>

THANK YOU!